

# Data Analysis with Python 2024–2025

## Parte 3: NumPy (Parte 2)

### Soluções dos Exercícios

Soluções independentes elaboradas para fins de estudo, com base nos enunciados originais

### Nota Importante

Este material é uma adaptação das soluções independentes para os exercícios baseados no curso *Data Analysis with Python 2024-2025*, oferecido pela **The University of Helsinki MOOC Center**. As soluções abaixo não são o gabarito oficial do curso.

### Conteúdo

## 1 Soluções - Capítulo 3 (Parte 2)

### Exercício 3.1 – Comparação de colunas

#### Solução proposta

```
import numpy as np

def column_comparison(a):
    return a[a[:, 1] > a[:, -2]]
```

A comparação é vetorizada e usa uma máscara booleana para selecionar as linhas desejadas.

### Exercício 3.2 – Primeira metade e segunda metade

#### Solução proposta

```
import numpy as np

def first_half_second_half(a):
    m = a.shape[1] // 2
    left_sum = np.sum(a[:, :m], axis=1)
    right_sum = np.sum(a[:, m:], axis=1)
    return a[left_sum > right_sum]
```

As duas somas são calculadas exatamente uma vez cada.

### Exercício 3.3 – Mais frequente primeiro

#### Solução proposta

```
import numpy as np

def most_frequent_first(a, c):
    values, counts = np.unique(a[:, c], return_counts=True)
    frequency = dict(zip(values.tolist(), counts.tolist()))
    row_counts = np.array([frequency[x] for x in a[:, c]])
    order = np.argsort(-row_counts, kind="stable")
    return a[order]
```

A coluna escolhida é usada apenas para calcular a frequência de cada valor; o array inteiro é então reordenado de acordo com essas frequências.

### Exercício 3.4 – Potência de matriz

#### Solução proposta

```
import numpy as np
from functools import reduce

def matrix_power(a, n):
    if n == 0:
        return np.eye(a.shape[0])
    base = np.linalg.inv(a) if n < 0 else a
    exponent = abs(n)
    return reduce(lambda x, y: x @ y, (base for _ in range(exponent)))
```

A expressão geradora fornece a matriz repetidamente ao `reduce`, e a multiplicação utilizada é a multiplicação matricial.

### Exercício 3.5 – Correlação

#### Solução proposta

##### Parte 1

Assumindo que a função `load()` devolva o array Iris:

```
from scipy.stats import pearsonr

def lengths():
    data = load()
    return pearsonr(data[:, 0], data[:, 2])[0]
```

##### Parte 2

```
import numpy as np

def correlations():
    data = load()
    return np.corrcoef(data, rowvar=False)

# Para identificar o par mais correlacionado sem considerar a
# diagonal principal:
def most_correlated_pair():
    data = load()
    corr = np.corrcoef(data, rowvar=False)
    upper = np.triu(np.abs(corr), k=1)
    i, j = np.unravel_index(np.argmax(upper), upper.shape)
    return i, j, corr[i, j]
```

No conjunto Iris clássico, o par mais fortemente correlacionado é o comprimento da pétala e a largura da pétala.

### Exercício 3.6 – Encontro de retas

#### Solução proposta

Para  $y = a_1x + b_1$  e  $y = a_2x + b_2$ , podemos escrever  $a_1x - y = -b_1$  e  $a_2x - y = -b_2$ .

```
import numpy as np

def meeting_lines(a1, b1, a2, b2):
    A = np.array([[a1, -1], [a2, -1]], dtype=float)
    b = np.array([-b1, -b2], dtype=float)
    x, y = np.linalg.solve(A, b)
    return x, y
```

### Exercício 3.7 – Encontro de planos

#### Solução proposta

As equações  $z = a_iy + b_ix + c_i$  podem ser reescritas como  $b_ix + a_iy - z = -c_i$ .

```
import numpy as np

def meeting_planes(a1, b1, c1, a2, b2, c2, a3, b3, c3):
    A = np.array([
        [b1, a1, -1],
        [b2, a2, -1],
        [b3, a3, -1]
    ], dtype=float)
    b = np.array([-c1, -c2, -c3], dtype=float)
    x, y, z = np.linalg.solve(A, b)
    return x, y, z
```

### Exercício 3.8 – Retas quase concorrentes

#### Solução proposta

Podemos montar o mesmo sistema do exercício 3.6 e resolvê-lo com mínimos quadrados. Quando o sistema possui uma solução exata,  $A @ \text{solution}$  coincide com  $b$ ; caso contrário, a solução é o ponto que minimiza o erro quadrático.

```
import numpy as np

def almost_meeting_lines(a1, b1, a2, b2):
    A = np.array([[a1, -1], [a2, -1]], dtype=float)
    b = np.array([-b1, -b2], dtype=float)
    solution, residuals, rank, _ = np.linalg.lstsq(A, b, rcond=None)
    exact = (rank == 2) and np.allclose(A @ solution, b)
    return solution, exact
```

**Créditos:** Este conteúdo foi originalmente criado pela *The University of Helsinki MOOC Center* para o curso *Data Analysis with Python*.